Tuning Guide

This guide is meant to assist users in tuning their navigation system. While Configuration Guide is the home of the list of parameters for all of Nav2, it doesn't contain much color for how to tune a system using the most important of them. The aim of this guide is to give more advice in how to setup your system beyond a first time setup, which you can find at First-Time Robot Setup Guide. This will by no means cover all of the parameters (so please, do review the configuration guides for the packages of interest), but will give some helpful hints and tips.

This tuning guide is a perpetual work in progress. If you see some insights you have missing, please feel free to file a ticket or pull request with the wisdom you would like to share. This is an open section supported by the generosity of Nav2 users and maintainers. Please consider paying it forward.

Inflation Potential Fields

Many users and ecosystem navigation configuration files the maintainers find are really missing the point of the inflation layer. While it's true that you can simply inflate a small radius around the walls to weight against critical collisions, the true value of the inflation layer is creating a consistent potential field around the entire map.

Some of the most popular tuning guides for ROS Navigation / Nav2 even call this out specifically that there's substantial benefit to creating a gentle potential field across the width of the map - after inscribed costs are applied - yet very few users do this in practice.

This habit actually results in paths produced by NavFn, Theta*, and Smac Planner to be somewhat suboptimal. They really want to look for a smooth potential field rather than wide open 0-cost spaces in order to stay in the middle of spaces and deal with close-by moving obstacles better. It will allow search to be weighted towards freespace far before the search algorithm runs into the obstacle that the inflation is caused by, letting the planner give obstacles as wide of a berth as possible.

So it is the maintainers' recommendation, as well as all other cost-aware search planners available in ROS, to increase your inflation layer cost scale and radius in order to adequately produce a smooth potential across the entire map. For very large open spaces, its fine to have 0-cost areas in the middle, but for halls, aisles, and similar; **please create a smooth potential to provide the best performance**.

Robot Footprint vs Radius

Nav2 allows users to specify the robot's shape in 2 ways: a geometric `footprint` or the radius of a circle encompassing the robot. In ROS (1), it was pretty reasonable to always specify a radius approximation of the robot, since the global planning algorithms didn't use it and the local planners / costmaps were set up with the circular assumption baked in.

However, in Nav2, we now have multiple planning and controller algorithms that make use of the full SE2 footprint. If your robot is non-circular, it is recommended that you give the planners and controllers the actual, geometric footprint of your robot. This will allow the planners and controllers to plan or create trajectories into tighter spaces. For example, if you have a very long but skinny robot, the circular assumption wouldn't allow a robot to plan into a space only a little wider than your robot, since the robot would not fit length-wise.

The kinematically feasible planners (e.g. Smac Hybrid-A*, Smac State Lattice) will use the SE2 footprint for collision checking to create kinematically feasible plans, if provided with the actual footprint. As of December, 2021 all of the controller plugins support full footprint collision checking to ensure safe path tracking. If you provide a footprint of your robot, it will be used to make sure trajectories are valid and it is recommended you do so. It will prevent a number of "stuck robot" situations that could have been easily avoided.

If your robot is truly circular, continue to use the `robot_radius` parameter. The three valid reasons for a non-circular robot to use the radius instead:

- The robot is very small relative to the environment (e.g. RC car in a warehouse)
- The robot has such limited compute power, using SE2 footprint checking would add too much computational burden (e.g. embedded micro-processors)
- If you plan to use a holonomic planner (e.g. Theta*, Smac 2D-A*, or NavFn), you may continue to use the circular footprint, since these planners are not kinematically feasible and will not make use of the SE2 footprint anyway.

Rotate in Place Behavior

Using the Rotation Shim Controller, a robot will simply rotate in place before starting to track a holonomic path. This allows a developer to tune a controller plugin to be optimized for path tracking and give you clean rotations, out of the box.

This was added due to quirks in some existing controllers whereas tuning the controller for a task can make it rigid - or the algorithm simply doesn't rotate in place when working with holonomic paths (if that's a desirable trait). The result is an awkward, stuttering, or whipping around behavior when your robot's initial and path heading's are significantly divergent. Giving a controller a better starting point to start tracking a path makes tuning the controllers significantly easier and creates more intuitive results for on-lookers (in one maintainer's opinion).

Note: If using a non-holonomic, kinematically feasible planner (e.g. Smac Hybrid-A*, Smac State Lattice), this is not a necessary behavioral optimization. This class of planner will create plans that take into account the robot's starting heading, not requiring any rotation behaviors.

This behavior is most optimally for:

- Robots that can rotate in place, such as differential and omnidirectional robots.
- Preference to rotate in place when starting to track a new path that is at a significantly different heading than the robot's current heading – or when tuning your controller for its task makes tight rotations difficult.

Planner Plugin Selection

Nav2 provides a number of planning plugins out of the box. For a first-time setup, see Setting Up Navigation Plugins for a more verbose breakdown of algorithm styles within Nav2, and Navigation Plugins for a full accounting of the current list of plugins available (which may be updated over time).

In general though, the following table is a good guide for the optimal planning plugin for different types of robot bases:

Plugin Name	Supported Robot Types
NavFn Planner	Circular Differential, Circular Omnidirectional
Smac Planner 2D	
Theta Star Planner	
Smac Hybrid-A* Planner	Non-circular or Circular Ackermann, Non-circular or Circular Legged
Smac Lattice Planner	Non-circular Differential, Non-circular Omnidirectional, Arbitrary

If you are using a non-circular robot with very limited compute, it may be worth assessing the benefits of using one of the holonomic planners (e.g. particle assumption planners). It is the recommendation of the maintainers to start using one of the more advanced algorithms appropriate for your platform *first*, but to scale back the planner if need be. The run-time of the feasible planners are typically on par (or sometimes faster) than their holonomic counterparts, so don't let the more recent nature of them fool you.

Since the planning problem is primarily driven by the robot type, the table accurately summarizes the advice to users by the maintainers. Within the circular robot regime, the choice of planning algorithm is dependent on application and desirable behavior. NavFn will typically make broad, sweeping curves; Theta* prefers straight lines and supports them at any angle; and Smac 2D is essentially a classical A* algorithm with cost-aware penalties.

> **Note**
>
> These are simply the default and available plugins from the community. For a specific application / platform, you may also choose to use none of these and create your own, and that's the intention of the Nav2 framework. See the Writing a New Planner Plugin tutorial for more details. If you're willing to contribute this work back to the community, please file a ticket or contact a maintainer! They'd love to hear from you.

Controller Plugin Selection

Nav2 provides a number of controller plugins out of the box. For a first-time setup, see Setting Up Navigation Plugins for a more verbose breakdown of algorithm styles within Nav2, and Navigation Plugins for a full accounting of the current list of plugins available (which may be updated over time).

In general though, the following table is a good first-order description of the controller plugins available for different types of robot bases:

Plugin Name	Supported Robot Types	Task
DWB controller	Differential, Omnidirectional	Dynamic obstacle avoidance Dynamic obstacle avoidance
MPPI Controller	Differential, Omnidirectional, Ackermann, Legged	
RPP controller	Differential, Ackermann, Legged	Exact path following
Rotation Shim	Differential, Omnidirectional	Rotate to rough heading
VP controller	Differential, Ackermann, Legged	High speed path tracking

All of the above controllers can handle both circular and arbitrary shaped robots in configuration.

Regulated Pure Pursuit is good for exact path following and is typically paired with one of the kinematically feasible planners (eg State Lattice, Hybrid-A*, etc) since those paths are known to be drivable given hard physical constraints. However, it can also be applied to differential drive robots who can easily pivot to match any holonomic path. This is the plugin of choice if you simply want your robot to follow the path, rather exactly, without any dynamic obstacle avoidance or deviation. It is simple and geometric, as well as slowing the robot in the presence of near-by obstacles *and* while making sharp turns.

DWB and MPPI are both options that will track paths, but also diverge from the path if there are dynamic obstacles present (in order to avoid them). DWB uses this heavily scoring multiple trajectories on a set of critics. These trajectories are also generated via plugins that can be replaced, but support out of the box Omni and Diff robot types within the valid velocity and acceleration restrictions. These critics are plugins that can be selected at run-time and contain weights that may be tuned to create the desired behavior, such as minimizing path distance, minimizing distance to the goal or headings, and other action penalties that can be designed. This does require a bit of tuning for a given platform, application, and desired behavior, but it is possible to tune DWB to do nearly any single thing well.

MPPI on the other hand implements an optimization based approach, using randomly perturbed samples of the previous optimal trajectory to maximize a set of plugin-based objective functions. In that regard, it is similar to DWB however MPPI is a far more modern and advanced technique that will deal with dynamic agents in the environment and create intelligent behavior due to the optimization based trajectory planning, rather then DWB's constant action model. MPPI however does have moderately higher compute costs, but it is highly recommended to go this route and has received considerable development resources and attention due to its power. This typically works pretty well out of the box, but to tune for specific behaviors, you may have to retune some of the parameters. The README.md file for this package contains details on how to tune it efficiently.

The Rotation Shim Plugin helps assist plugins like TEB and DWB (among others) to rotate the robot in place towards a new path's heading before starting to track the path. This allows you to tune your local trajectory planner to operate with a desired behavior without having to worry about being able to rotate on a dime with a significant deviation in angular distance over a very small euclidean distance. Some controllers where heavily tuned for accurate path tracking are constrained in their actions and don't very cleanly rotate to a new heading. Other controllers have a 'spiral out' behavior because their sampling requires some translational velocity, preventing it from simply rotating in place. This helps alleviate that problem and makes the robot rotate in place very smoothly.

Finally, Vector Pursuit is another good path tracking solution and just like RPP, is paired with a kinematically feasible planner. It is a bit more advanced than RPP in the sense it also takes path heading into account. Vector Pursuit can handle complex paths at high speeds, but it is still a simple geometric controller thus requiring low computation resources.

> **Note**
>
> These are simply the default and available plugins from the community. For a specific robot platform / company, you may also choose to use none of these and create your own. See the Writing a New Controller Plugin tutorial for more details. If you're willing to contribute this work back to the community, please file a ticket or contact a maintainer! They'd love to hear from you.

Caching Obstacle Heuristic in Smac Planners

Smac's Hybrid-A* and State Lattice Planners provide an option, `cache_obstacle_heuristic`. This can be used to cache the heuristic to use between replannings to the same goal pose, which can increase the speed of the planner **significantly** (40-300% depending on many factors). The obstacle heuristic is used to steer the robot into the middle of spaces, respecting costs, and drives the kinematically feasible search down the corridors towards a valid solution. Think of it like a 2D cost-aware search to "prime" the planner about where it should go when it needs to expend more effort in the fully feasible search / SE2 collision checking.

This is useful to speed up performance to achieve better replanning speeds. However, if you cache this heuristic, it will not be updated with the most current information in the costmap to steer search. During planning, the planner will still make use of the newest cost information for collision checking, *thusly this will not impact the safety of the path*. However, it may steer the search down newly blocked corridors or guide search towards areas that may have new dynamic obstacles in them, which can slow things down significantly if entire solution spaces are blocked.

Therefore, it is the recommendation of the maintainers to enable this *only* when working in largely static (e.g. not many moving things or changes, not using live sensor updates in the global costmap, etc) environments when planning across large spaces to singular goals. Between goal changes to Nav2, this heuristic will be updated with the most current set of information, so it is not very helpful if you change goals very frequently.

Costmap2D Plugins

Costmap2D has a number of plugins that you can use (including the availability for you to create your own!).

- `StaticLayer`: Used to set the map from SLAM (either pre-built or currently being built live) for sizing the environment and setting key features like walls and rooms for global planning.
- `InflationLayer`: Inflates with an exponential function the lethal costs to help steer navigation algorithms away from walls and quickly detect collisions
- `ObstacleLayer`: 2D costmap layer used for 2D planar lidars or when on low-compute devices where 3D layer cannot be used on 3D sensors. If you have enough compute, use the `VoxelLayer` for 3D data which provides more accurate results.
- `VoxelLayer`: 3D costmap layer for non-planar 2D lidars or depth camera processing based on raycast clearing. Not suitable for 3D lidars due to sparseness of data collected.
- `SpatioTemporalVoxelLayer`: 3D costmap layer for 3D lidars, non-planar 2D lidars, or depth camera processing based on temporal decay. Useful for robots with high sensor coverage like 3D lidars or many depth cameras at a reduced computational overhead due to lack of raycasting.
- `RangeLayer`: Models sonars, IR sensors, or other range sensors for costmap inclusion
- `DenoiseLayer`: Removes salt and pepper noise from final costmap in order to remove unfiltered noise. Also has the option to remove clusters of configurable size to remove effects of dynamic obstacles without temporal decay.
- `PluginContainerLayer`: Combines the costmap layers specified within this plugin, resulting in an internal costmap that is a product of the costmap layers specified under this layer. This would allow different isolated combinations of costmap layers within the same parent costmap, such as applying a different inflation layers to static layers and obstacle layers

In addition, costmap filters: - `KeepoutFilter`: Marks keepout, higher weighted, or lower weighted zones in the costmap - `SpeedFilter`: Reduces or increases robot speeds based on position - `BinaryFilter`: Enables or disables a binary topic when in particular zones

Note: When the costmap filters can be paired with the `VectorObject` server to use vectorized zones rather than map rastered zones sharing the same software.

Nav2 Launch Options

Nav2's launch files are made to be very configurable. Obviously for any serious application, a user should use `nav2_bringup` as the basis of their navigation launch system, but should be moved to a specific repository for a users' work. A typical thing to do is to have a `<robot_name>_nav` configuration package containing the launch and parameter files.

Within `nav2_bringup`, there is a main entryfile `tb3_simulation_launch.py`. This is the main file used for simulating the robot and contains the following configurations:

- `slam`: Whether or not to use AMCL or SLAM Toolbox for localization and/or mapping. Default `false` to AMCL.
- `map`: The filepath to the map to use for navigation. Defaults to `map.yaml` in the package's `maps/` directory.
- `world`: The filepath to the world file to use in simulation. Defaults to the `worlds/` directory in the package.
- `params_file`: The main navigation configuration file. Defaults to `nav2_params.yaml` in the package's `params/` directory.
- `autostart`: Whether to autostart the navigation system's lifecycle management system. Defaults to `true` to transition up the Nav2 stack on creation to the activated state, ready for use.
- `use_composition`: Whether to launch each Nav2 server into individual processes or in a single composed node, to leverage savings in CPU and memory. Default `true` to use single process Nav2.
- `use_respawn`: Whether to allow server that crash to automatically respawn. When also configured with the lifecycle manager, the manager will transition systems back up if already activated and went down due to a crash. Only works in non-composed bringup since all of the nodes are in the same process / container otherwise.
- `use_sim_time`: Whether to set all the nodes to use simulation time, needed in simulation. Default `true`.
- `rviz_config_file`: The filepath to the rviz configuration file to use. Defaults to the `rviz/` directory's file.
- `use_simulator`: Whether or not to start the Gazebo simulator with the Nav2 stack. Defaults to `true` to launch Gazebo.
- `use_robot_state_pub`: Whether or not to start the robot state publisher to publish the robot's URDF transformations to TF2. Defaults to `true` to publish the robot's TF2 transformations.
- `use_rviz`: Whether or not to launch rviz for visualization. Defaults to `true` to show rviz.
- `headless`: Whether or not to launch the Gazebo front-end alongside the background Gazebo simulation. Defaults to `true` to display the Gazebo window.
- `namespace`: The namespace to launch robots into.
- `robot_name`: The name of the robot to launch.
- `robot_sdf`: The filepath to the robot's gazebo configuration file containing the Gazebo plugins and setup to simulate the robot system.
- `x_pose`, `y_pose`, `z_pose`, `roll`, `pitch`, `yaw`: Parameters to set the initial position of the robot in the simulation.

Other Pages We'd Love To Offer

If you are willing to chip in, some ideas are in https://github.com/ros-navigation/docs.nav2.org/issues/204, but we'd be open to anything you think would be insightful!